Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled "Core JDO" are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdobook@yahoogr oups.com

CHAPTER 10

"You can tell a lot about a fellow's character by his way of eating jellybeans"
-Ronald Reagan quoted in Observer, March 29 1981

JDO and Enterprise JavaBeans (EJB)

This chapter will attempt to give you a better understanding of Enterprise JavaBeans (EJB) in the context of JDO. The goal is not to simply show 'how' to code EJB and JDO together, but to also talk about the 'why' and 'when' which should allow you to ultimately decide 'if' of the combination of these two powerful Java APIs makes sense.

More or less complete code examples will be shown to clarify the points discussed. Brief summaries of EJB concepts will be presented, however a certain familiarity of the reader with the basic EJB architecture is assumed. This chapter is not intended to be a comprehensive tutorial on EJB's, several good ones already exist such as the J2EE tutorial from Sun at http://java.sun.com/j2ee/tutorial.

The example code will not show home- and remote interfaces or deployment descriptors and instead focus on relevant code snippets from within the bean implementation classes themselves.

INTRODUCTION

The Enterprise JavaBeans specification defines EJB as follows: "The Enterprise JavaBeans architecture is a component architecture for the development and deployment of component-based distributed business applications. Enterprise beans are components of distributed transaction-oriented enterprise applications. Applications written using the Enterprise JavaBeans architecture are scalable, transactional, and multi-user secure. These applications may be written once, and then deployed on any server platform that supports the Enterprise JavaBeans specification."

In practice, this translates into a number of concrete features:

- Transaction management (declarative or explicit via code, possibly distributed)
- Contracts (components, remote and local clients) & Views (incl. Web Services)
- Simple Security management (declarative or explicit)
- Instance and resource pooling
- Exception handling (standard conventions)
- Deployment (packaging, descriptors)
- Management (run-time monitoring)

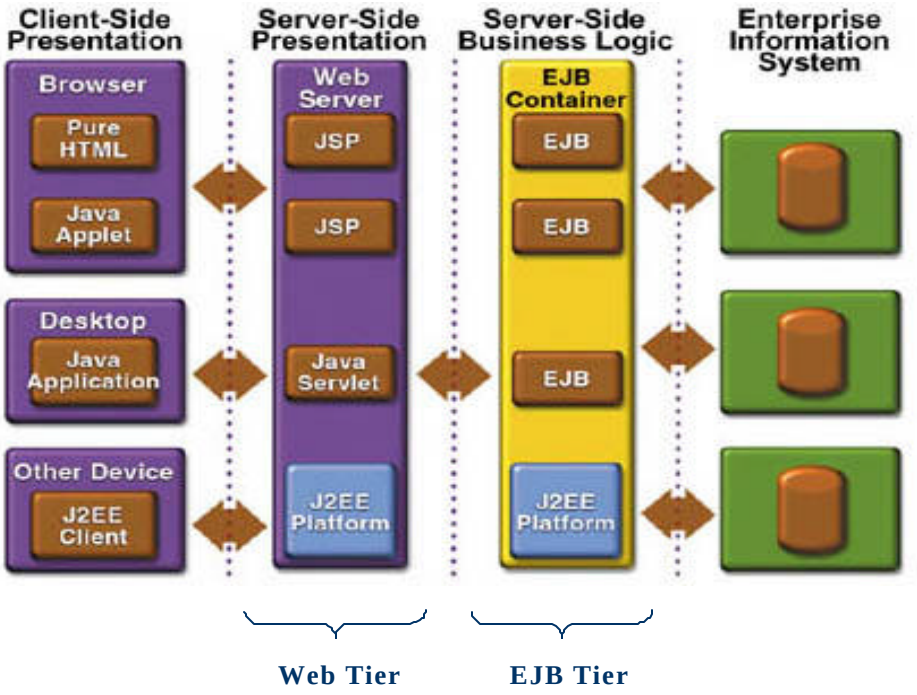

Figure 10-1 EJBs as server-side business logic in the classical J2EE Architecture

There are currently three types of enterprise beans that model typical but fairly different and technically relatively unrelated requirements of modern enterprise software:

- Session Beans: Components that represent stateless services which will be invoked synchronously from remote clients. They are possibly accessible as web

service endpoint views. A stateful variation models components that represent a conversational session with a particular client. Session beans are accessible with remote invocation or via local interfaces.

- Message-driven beans: Components that represent a stateless service and whose invocation is asynchronous, usually driven by the arrival of Java Messaging Service (JMS) messages.
- Entity beans: Components that represent persistent objects. Entity beans are also accessible with remote invocation or via local interfaces. Note that entity beans are technically unrelated to JDO and predate it; more on this later

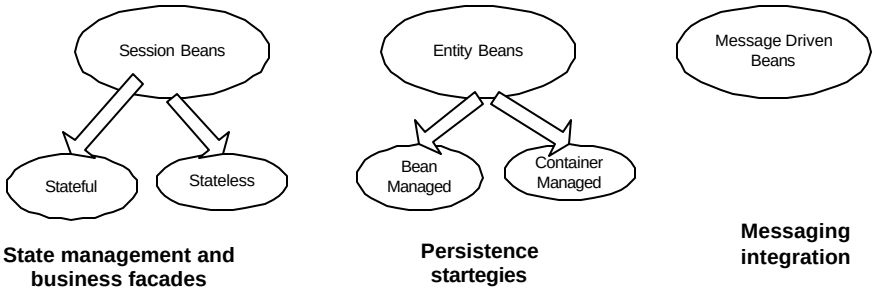

Figure 10-2 Types of EJB components

Out of these three types, one is related to persistence and thus overlaps with JDO. The two other types of EJBs represent service-oriented interfaces which in order to do their task often rely on and thus interface with a persistence API, such as JDBC, entity beans, JDO, or others. The point to remember is that these three enterprise bean types do not depend on each other in any way; but offer different features. We'll look at the role of JDO with each throughout this chapter.

SESSION BEANS AND JDO

A typical session bean has the following characteristics, according to the EJB specification:

- "Executes on behalf of a single client"
- "Can be transaction-aware"
- "Often updates shared data in an underlying database" but "Does not represent directly shared data in the database, although it may access and update such data"

- "Is relatively short-lived" and "A typical EJB Container provides a scalable runtime environment to execute a large number of session objects concurrently."

The EJB specification defines stateless session beans, which do no keep state between business method invocations; as well as stateful session beans which can keep conversational state between business method invocations. In short, a stateful session bean holds client specific state in the business tier.

Additionally, such beans can rely on two types of transaction demarcation mechanisms:

Container Managed Transactions (CMT) where the EJB container coordinates transactions across business method calls. A bean using CMT declares in its XML deployment descriptor its transactional properties.

Bean Management Transactions (BMT) where a bean's implementation itself controls transactions via explicit code, i.e. begins, commits respectively rollbacks transactions.

The choice of stateless versus stateful and the transaction management schema is technically unrelated and orthogonal to each other; this thus leads to four possible scenarios:

- Stateless Session Bean with Container Managed Transactions
- Stateful Session Bean with Container Managed Transactions
- Stateless Session Bean with Bean Managed Transactions
- Stateful Session Bean with Bean Managed Transactions

The first blend (Stateless Session Bean with Container Managed Transactions) is generally the most useful and widely used, as it combines managed transactions with the stateless architecture. The stateless architecture best allows EJB containers to efficiently implement remote request load-balancing and fail-over features by reusing and sharing beans. Together, these are two of the major advantages of using EJBs over plain simple vanilla Java classes to implement business logic.

Subsequently we'll first examine some more details regarding transaction management, and then look at some concrete code specifically for the case of Stateless Session Bean with Container Managed Transactions and Stateful Session Bean with Container Managed Transactions.

Transaction Management

Previous chapters of this book (including JCA and Transactions chapters) already provided the background on the issue of managed versus unmanaged JDO environments with its different transaction mechanisms.

As a brief reminder, the transaction demarcation mechanism in a non-managed JDO environment uses the `javax.jdo.Transaction` interface returned by the `PersistenceManager.currentTransaction()` method. In a managed environment however, a JDO application relies on the standard J2EE `javax.transaction.UserTransaction` from the Java Transaction API, JTA.

Bean Managed Transactions (BMT) with JDO Transaction

When building session beans using Bean Managed Transactions, both the JDO and the JTA transaction demarcation API can technically be used.

Should the choice be to use the `javax.jdo.Transaction`, then the code within a session bean's business method would not really differ much from standalone simple JDO code seen elsewhere in this book. So in a business method in a Session Bean with BMT one would simply say:

```
PersistenceManager pm = pmf.getPersistenceManager();

// No J2EE JTA transaction can be active at this point!
pm.currentTransaction().begin();

// Do something using JDO now...

pm.currentTransaction().commit();
pm.close();
```

This architecture could make sense for simple BMT session beans which do not require the transaction context to be propagated when other methods are called from it.

If an enterprise component does require transaction propagation however, it should either use the JTA API for explicit demarcation, or straight use CMT for implicit declarative demarcation. Both approaches will be illustrated in the following paragraphs.

Bean Managed Transactions (BMT) with JTA Transaction

As has been shown in the JCA and Transaction chapters, a JDO implementation which claims JCA-compliance (sometimes referred to as "n-tier ready", "supports JDO managed-environment", "supports application server environment" or "enterprise integration enabled" in documentation) is able to perfectly blend into the J2EE and thus EJB environment. In this approach, a BMT-based business method would use a `javax.transaction.UserTransaction`, obtained via the EJB `SessionContext.getUserTransaction()` method, for transaction demarcation, as any non-JDO related session bean would. The only condition for this to be possible is that it is crucial to request the `PersistenceManager` from the `PersistenceManagerFactory` after having initiated a JTA transaction (BMT-scenario), or alternatively ensuring that code executes within an elsewhere initiated currently active JTA transaction (CMT-scenario, see below). Respective code from a BMT session bean business method would thus look something like this:

```
sessionContext.getUserTransaction().begin();
PersistenceManager pm = pmf.getPersistenceManager();

// Do something using JDO now...

pm.close();
sessionContext.getUserTransaction().commit();
```

This allows a JDO implementation to implement `PersistenceManagerFactory.getPersistenceManager()` such that it returns a `PersistenceManager` associated with the `javax.transaction.Transaction` of the executing thread. If datastore transactions are used by JDO, the underlying datastore connection will also be enlisted in the transaction.[1]

This returned PM will either be a new one just created, set up with the appropriate synchronizations and enlisted in the respective connection, as in the above example where a user transaction was just initiated by the bean. If however a user transaction was already active and the BMT bean business method did not explicitly demarcate a transaction beginning, then a JDO implementation will ensure that the PM is an existing one currently correctly enlisted in the transaction. This is of particular importance for CMT as well, as below.

Each EJB business method should obtain its persistence manager from the persistence manager factory and close the persistence manager before returning, as shown in the above

[1] If optimistic JDO transactions and a transactional in-memory cache are used, as opposed to datastore transactions, then there may not actually be an underlying datastore transaction associated. Note that it is still possible to use JDO optimistic transactions even when using JTA user transactions, by specifying `setOptimistic(true)` on the `PersistenceManagerFactory` instead of the `javax.jdo.Transaction` class which would not be used anymore in this scenario.

code snippet. This is not strictly necessary for stateful session beans under BMT, but the implied holding of a reference to a persistence manager and user transaction in an instance variable is a dubious practice. In fact, as we have just said, a client is guaranteed to obtain a PM enlisted in the same user transaction, if it is still active. Obtaining a persistence manager and closing it before returning in each EJB business method is thus perfectly safe and preferred practice, and causes no issues for transactions that span multiple bean method invocations.

Container Managed Transactions (CMT) via JTA

CMT enterprise beans can use neither the JDO transaction demarcation `javax.jdo.Transaction`, nor the JTA user transaction management methods (`begin()`, `commit()`, `rollback()`) in their code. On the contrary, their business methods are guaranteed to be called within an active JTA transaction, if so declared. CMT relies on the container, other enterprise beans, or external plain Java classes for transaction initiation via the `javax.transaction.UserTransaction` interface.

As discussed above, this guarantees that the JDO `PersistenceManager` obtained in a business method of a CMT enterprise bean will always be correctly associated and enlisted in the respective J2EE transaction.

When using CMT with declarative transaction management via the EJB deployment descriptor, the respective attributes in the XML descriptors need to be understood and set correctly, as per the EJB specification. The intended configuration usually is one of the following:

- `Required`: automatically begins new transaction if none, and in this case commits before returning to the caller.
- `Mandatory`: if existing then as in `Required`, else throw Exception.
  Never automatically begin a transaction.
- `Never`: Always to be called without a transactional context, if existing then throw Exception. Could potentially be used together with JDO `NontransactionalRead` in respective methods.

These configurations would generally be more exotic and less used:

- `NotSupported`: “Unspecified transaction context”. If existing then suspend.
- `Supports`: Like `Required` if existing, like `NotSupported` if none.
- `RequiresNew`: if none than begin new transaction and commit before return. If existing, then suspend.

In summary, it can be said that transaction management with enterprise session beans that rely on JDO for persistence matters really does not have to be any different from how EJB transaction demarcation has always been.

The remainder of this chapter focuses on examples of how to use JDO from CMT session beans in order to keep transaction management out of sample code.

Stateless Session Bean using CMT Example

Without further ado, let's dive into practice and look at how a full Stateless Session Bean with Container Managed Transaction would be coded.

The class would be declared like this:

```
public class ExampleCMTBeanWithJDO implements SessionBean {
    private javax.ejb.SessionContext ejbCtx;
    private PersistenceManagerFactory jdoPMF;
    private InitialContext jndiInitCtx;
    // No other instance variables in a stateless bean!

    private final static String jndiName =
                          "java:comp/env/jdo/bookstorePMF";
```

In `setSessionContext()` the JDO `PersistenceManagerFactory` is acquired from JDNI and assigned to an instance variable of the session bean:

```
public void setSessionContext(SessionContext sessionCtx)
                                      throws EJBException {
        ejbCtx = sessionCtx;
        try {
            jndiInitialContext = new InitialContext();
            Object o = jndiInitCtx.lookup(jndiName);

            jdoPMF = (PersistenceManagerFactory)
                        PortableRemoteObject.narrow (o,
                          PersistenceManagerFactory.class);

        } catch (NamingException ex) {
            throw new EJBException(ex);
        }
    }
```

The other EJB service methods (`ejbCreate()`, `ejbRemove()`, `ejbActivate()` and `ejbPassivate()`) will usually stay empty, unless code is required for other (non-JDO related) purposes in business methods later:

```
public void ejbCreate() throws javax.ejb.CreateException {
}

public void ejbRemove() throws EJBException {
}

public void ejbActivate() throws EJBException {
}

public void ejbPassivate() throws EJBException {
}
```

Now in the business methods of the EJB, JDO can be easily used according to this generic code template (ignore return and argument types for now, more on that later):

```
public void doSomething(int arg) {
        PersistenceManager pm;
        try {
            pm = jdoPMF.getPersistenceManager();

            //
            // Do something using JDO now...
            //

        } catch (Exception ex) {
            ejbCtx.setRollbackOnly();    // sic!
            throw new EJBException(ex);
        } finally {
            try {
                if (pm != null && !pm.isClosed())
                    pm.close();
            } catch (Exception ex) {
                // Log it
            }
        }
        // Maybe, return something;
    }
```

The core of the business logic, what it actually does, goes in the middle of the above method, after the `getPersistenceManager()`. We'll look a little closer at the details of such code, and any related JDO specifics, just below.

How did the JDO PMF get into JNDI?

In the code above, the `PersistenceManagerFactory` is obtained via a lookup from JNDI, instead of using the `JDOHelper` as in the simpler examples elsewhere in this book. How, when and by who did that PMF get bound into JNDI in the first place?

The idea in line with global J2EE architecture is that the JDO PMF is bound into JDNI by a JCA adaptor; more details on this can be found in the chapter dedicated to JDO and JCA, as well as in the Transactions chapter.

Alternatively, a simple plain vanilla Java "start-up class" could achieve a similar goal, by manually creating a `PersistenceManagerFactory`, presumably using the `JDOHelper` class, and binding it into JNDI. However, J2EE "start-up" classes are a non-standard feature that depends on respective proprietary Application Server APIs. Furthermore, and more importantly, such a manual approach would defeat the purpose of the JCA architecture that JDO can leverage in a managed scenario, and would make it difficult to use Container Managed (possibly distributed) Transactions.

Stateful Session Bean with CMT Example

As mentioned in the introduction, the J2EE specification provides for another type of session bean, the stateful session bean, which automatically maintains its conversational state across multiple client-invoked methods.

Many business processes have simple conversations that can be modeled as completely stateless services via stateless session beans as already seen. Alternatively, if the service appears stateful but is simple enough, passing state from the client to the session bean as argument to each business method is a possibility.

However, some use cases describe business processes that are inherently conversational and require multiple and possibly varying sequences of method calls to complete. Sometimes it makes more sense to model this as a stateful session bean. This can be more efficient than reconstructing state inside the bean from arguments each time, or retrieved from any persistent datastore. This book is not the place to outline such a choice in more detail. We will however examine how to build such a stateful session bean with access to JDO, should this be a preference over a completely stateless architecture for a given scenario.

As above for the Stateless Session Bean example, we'll focus on Container Managed Transactions only in this code. BMT would be equally possible and look as outlined earlier.

The declarations at the beginning of the bean implementation class are in essence largely similar to the stateless source code example above. The one important difference is

that, because this is a stateful session bean, we are now permitted to keep instance variables across method calls; for example:

```
private String bookISBN;
```

This instance variable could remember the customer ID across business method invocations. Remember that instance variable of stateful session beans need to be `Serializable`; so while an instance variable of type `String` or other simple type is just fine, holding on to an arbitrary persistent object would be asking for trouble and deserves a closer second look:

```
private Category currentCategory;  // bad!
```

Instead, if a stateful session bean indeed needs to hold on to the same persistent object between business method calls, it is generally preferable to keep the JDO object identity instead of the persistent object itself in an instance variable:

```
private Object catID;  // ok.
// ...

public void doOneThing(String name) {
        PersistenceManager pm;
        try {
            pm = jdoPMF.getPersistenceManager();
            // ...
            Category catPC = ... // e.g. from Query result
            catID = pm.getObjectId(catPC);
        }
        // ... Omitting error handling etc. – it's as above
        finally {
            pm.close();
        }
}

public void doAnotherThing() {
        PersistenceManager pm;
        try {
            pm = jdoPMF.getPersistenceManager();
            Category CatPC = pm.getObjectById(CatID, true);
            // ...
        }
        // ... Omitting error handling etc. – it's as above
```

```
        finally {
            pm.close();
        }
}
```

Another important point worth mentioning is that just because the enterprise bean is stateful, this does not imply it is good practice to keep a reference to a JDO `PersistenceManager` in an instance variable. Just like in the example above for a stateless session bean, the business methods of stateful session beans should still obtain a PM at the beginning of every method, and close it at the end. Transactional integrity can still be guaranteed and is unrelated to holding on to one and the same PM from different stateful entity bean method calls.

Service Oriented Architectures (SOA)

In the above examples we have only looked at the structure of session enterprise beans, and abstracted the specifics of a service's method contract and inner workings. We will examine those more closely here.

Let's first briefly think about what we mean by a Service. A service in this context is something phenomenally simple: It has a signature, and is called with arguments and returns something. More importantly however, a service in this context is usually a remote service, often referred to as a remote session "façade", because it fronts local code. Arguments and return values are thus passed by value, thus in a serialized form, not by reference.

Of course stateless session beans are not simply dumb remote procedure call (in Java i.e. RMI) endpoints; they include transaction context propagation, possible clustering, some security authentication mechanism, etc. However, the fundamental architecture is the one of a remote invocation, still.

A service thus exposes coarse grained Use Cases to the public. Sometimes such "services" offered by session beans have relatively straightforward incoming arguments and return simple outgoing results: For example a `createXYZ(String name)` method with simple arguments such as Strings for a name, and similarly, methods of type `double getXYZ(String code)` or something along those lines. Business methods with such signatures are easy to implement and should not require further discussion after what was presented so far.

However, some session beans, more often than not, will want to return (or receive as arguments in an `updateXYZ()`-style scenario) more complex types; for example, a `Book`, an `Author`, or other objects "inspired" by the persistence-capable objects from the actual underlying domain model. It is this second case that needs to be examined more closely in the context of JDO.

The point here is that in a Service Oriented Architecture (SOA) a client can not simply "navigate" to other objects of a domain model that were not explicitly returned by value. If a

service does not return the data a service invoker requires, then the service definition, the service implementation, and the service invoking client need to be changed.

In such a pure SOA architecture, clients of a session bean service (be it the Web tier or standalone EJB clients) can also not use the JDO API directly, as this would be a violation of the Service Oriented Architecture. Clients would only invoke well-defined services. The objects returned from a service would thus not have any JDO environment on the client. This immediately leads to the following question: What instances of which class, should a service return, and receive as arguments? We'll look at this in more detail in the following two sections.

Data-Transfer Objects (AKA Value Objects) and JDO PC Serialization

Let's first examine the case of returning objects from an enterprise session bean service business method. We'll look at the reverse case, passing objects as arguments to a service, in the next section only.

For the sake of an illustrative example, imagine a library application: Say the session bean outlined above had a method to return a collection of books; it could be objects that meet a certain query condition, such as "has copies that the currently authenticated user has borrowed", or it could be a given number of objects, starting from a given index, or anything else that requires the method to return a collection of persistent objects. For the initial discussion, let us assume that the persistent `Book` class does not have any Java references to other persistent objects and thus looks something like this:

```
public class Book {
   //
   // JDO persistent-capable class that will be enhanced!
   //
   private String author;
   private String isbn;

   public String getAuthor() { return author; }
   public void setAuthor(String _a) { author = _a; }

   public String getISBN() { return isbn; }
   public void setISBN(String _isbn) { isnb = _isbn; }
}
```

This is often overly simplistic for most real-world applications, but bear with it as a first example for good reasons; we will look into the matter of graphs of objects further down.

What one may be tempted to write at first, without further thought, is something like this: (Following code is intentionally wrong; see discussion below.)

```
public Collection getBooks(/* e.g. int start, int count */)
{
   PersistenceManager pm = null;

   // try/catch/finally error handling omitted, as above
   pm = jdoPMF.getPersistenceManager();

   Query q = pm.newQuery(Book.class);
   // q.setFilter("...");
   Collection results = (Collection)q.execute();

   pm.close();
   return results;  //  BAD.  WRONG.  See below.
}
```

This will however not work for several reasons:

- the persistence-capable class is not declared `serializable`
- persistent objects will be accessed by the container for serialization before returning from method, but after transaction has been committed (both CMT and BMT)
- Most importantly, the collection returned by `Query.execute()` is not `serializable`. Worse yet, it could be holding on to datastore resources such as cursors etc.

The last point can be easily addressed, so let's get this out of the way first. The following code addresses that issue by simply copying JDO query results into a JDK standard `Collection`, here an `ArrayList`. It is assumed that a filter expression has limited the size of the result collection to a "sensible" size:

```
public Collection getBooks(/* int start, int count* /) {
   PersistenceManager pm = null;
   List results = null;

   // try/catch/finally error handling omitted, as above
   pm = jdoPMF.getPersistenceManager();

   Query q = pm.newQuery(Book.class);
   // q.setFilter("...");
   Collection jdoResult = (Collection)q.execute();
   results = new ArrayList(jdoResult);
```

```
    q.close(jdoResult);

    pm.close();
    return results;
}
```

This still won't work though. As mentioned, the persistent objects are not `serializable` at this point. Because the `Book` instances contained in the `ArrayList` will "travel over the wire" to the remote client after the EJB business method returns, we do have to stick something `serializable` into the collection. There are two ways to achieve this:

- Hand-crafted (or code-generated) non-persistence capable but `serializable` helper classes specifically for transporting the data between tiers. Readers familiar with other literature will of course recognize this as a familiar architecture choice, with Sun's J2EE patterns literature referring to this concept as Value objects (VO), while other sources prefer the term of Data Transfer Object (DTO) for the same idea: Instead of returning a collection of persistence capable `Book` instances, we would create instances of new transfer objects in the service. We restrain from the initial temptation of making this `BookVO` class extend the `Book` class. Similarly, we do not use a constructor that takes the persistent object. Instead of using a Constructor taking individual arguments, a dedicated Assembler class could have been used as well. This ensures decoupling and avoids dependency, so that only the `Serializable` class will be required on the classpath of the EJB client, and the persistent `Book` class remains restricted to the server with access to the domain model.

```
public class BookVO implements Serializable {
  private String author;
  private String isbn;

   public BookVO(String bookAuthor, bookISBN) {
      author = bookAuthor;
      isbn = bookISBN;
   }

   public String getAuthor() { return author; }
   public String getISBN() { return isbn; }

   // Note: Delibarately no setters yet!  See below.
}
```

```
public Collection getBooks() {
   PersistenceManager pm = null;
   List results = null;
   try {
       pm = jdoPMF.getPersistenceManager();
       Query q = pm.newQuery(Book.class);
       // q.setFilter("...");
       Collection jdoResult = (Collection)q.execute();

       results = new ArrayList(jdoResult.size());
       Iterator it = results.iterator();
       while ( it.hasNext() ) {
           Book pcBook = (Book)it.next();
           BookVO dtoBook = new BookVO(
                pcBook.getAuthor(), pcBook.getISBN());

           results.add(dtoBook);
       }

       q.close(jdoResult);
   } catch (Exception ex) {
       ejbCtx.setRollbackOnly();    // sic!
       throw new EJBException(ex);
   } finally {
       try {
           if (pm != null && !pm.isClosed())
               pm.close();
       } catch (Exception ex) {
           // Log it
       }
   }
   return results;
}
```

- Alternatively, it is perfectly possible to actually tag the `Book` persistence capable class as `serializable` in its declaration. An interesting and useful feature of JDO in this respect is that persistence-capable classes are guaranteed to implement Java JDK Serialization in a manner such that enhanced classes can be serialized to and from non-enhanced classes; i.e. a correct `serialVersionUID` will be calculated by an enhancer, etc. It is thus perfectly possible to use an enhanced and thus persistence capable version of a class in the service implementation on a server, and the non-enhanced code of the same class on a client.

Alternatively, the same enhanced class could also be used on the client; it will de-serialize into an environment without JDO runtime, and thus be in transient state. Transient objects are guaranteed to behave like non-enhanced ones. An additional issue in this option as opposed to the one above is that the commit which the EJB container will perform before the return (in CMT, or the demarcation the bean would do in BMT) has possibly flushed the persistent objects. So if the container tries to access objects for serialization after the commit, it will get an exception since there is no active transaction. This can be addressed by a `makeTransient()` call. Related to this, if there was any chance that not all attributes are read into memory yet, maybe due to a pre-fetch limitation, then invoking `retrieve()` before `makeTransient()` will ensure that all persistent fields are loaded into the persistent object by the respective `PersistenceManager`. Here is an example code all of illustrating this together:

```
public class Book implements Serializable {
   ...

public Collection getBooks() {
   PersistenceManager pm = null;
   List results = null;
   try {
       pm = jdoPMF.getPersistenceManager();
       Query q = pm.newQuery(Book.class);
       // q.setFilter("...");
       Collection jdoResult = (Collection)q.execute();

       pm.retrieveAll(jdoResult);
       results = new ArrayList( jdoResult );
       pm.makeTransientAll(results);

       // Or use loop to iterated and individually
       // retrieve/makeTransient instead of xyzAll():
       //
       // results = new ArrayList( jdoResult.size() );
       // Iterator it = results.iterator();
       // while ( it.hasNext() ) {
       //    Book pcBook = (Book)it.next();
       //    pm.retrieve(pcBook);
       //    pm.makeTransient(pcBook);  // !
       //    results.add(pcBook);
       // }
```

```
            q.close(jdoResult);
        } catch (Exception ex) {
            ejbCtx.setRollbackOnly();    // sic!
            throw new EJBException(ex);
        } finally {
            try {
                if (pm != null && !pm.isClosed())
                    pm.close();
            } catch (Exception ex) {
                // Log it
            }
        }
        return results;
    }
```

Some of the arguments for choosing one or the other of above strategies include:

- While coding additional classes for the first option may seem like a disadvantage of that approach at first, the implied reduction of coupling between the domain model used in the session bean implementation based on JDO, and the client, usually a presentation tier, can be an advantage in the longer term.
- If a service returns a condensed summary view, an analysis, results of a calculation, or any other information not directly represented by an existing class of the domain model, a dedicated non persistence capable DTO class likely makes sense anyway.

If a business method does not return a collection of objects, but merely one specific persistent object, the same issues would arise and above the points would equally apply.

Let's now extend the domain model of our `Book` example slightly, but significantly. Let's assume an at first glance small enhancement requested is that Books are categorized hierarchically. We introduce a new Category class, as follows:

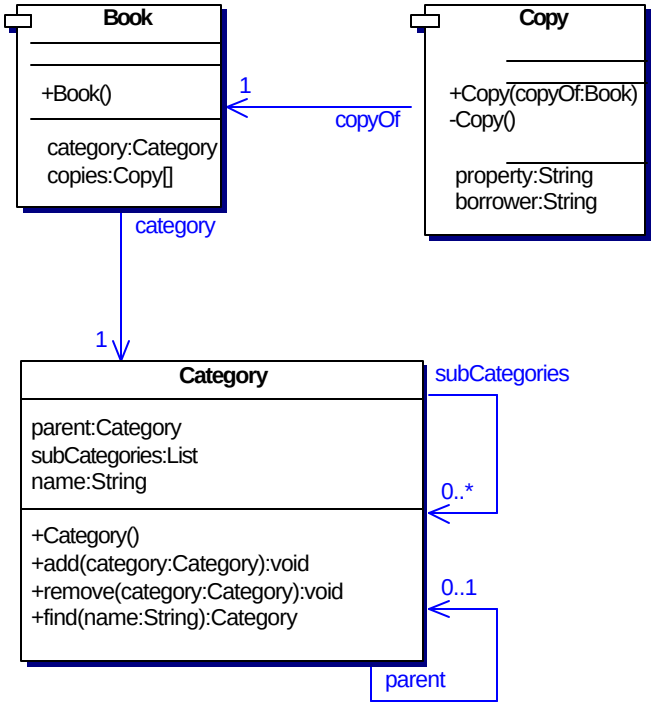

Figure 10-3 Extended example model, Book with association (UML representation)

A `Category` is just an example. What matters is that the persistent class `Book` now has a relationship to another persistent class. One can easily imagine other cases, indeed most real-world domain models while have various associations between persistent classes. Persistent objects thus are nodes in a graph, with links to other persistent objects.

And that's where it starts to get really interesting: With the approach of direct serialization of formerly persistent objects, which worked fine above for a simple case, we would now potentially serialize a huge data stream: The entire closure of instances would normally be serialized along.

Referenced instances would also have to be made transient; `retrieve()` and `makeTransient()` would have to be manually invoked for the entire graph of reachable instances, since that is what serialization will do. The `makeTransient()` call by itself is not propagated across the graph, although it would be possible to write graph traversal helper functions using reflection semi-automating such procedures.

However, the real question is: Do we actually want the entire Category hierarchy (examine how the Category instances themselves hold references to a parent and sub-

categories!) to be serialized and sent over the wire with each Book object? Presumably not, and the same would apply to other similarly structured real-world domain models.

It could seem that there are "work-arounds" for some of these issues, for example:

- Code could explicitly "nullify" references after making the instance transient
- Domain model could be changed in order to make it "serialization-friendly" by taking advantage of JDO's ability to declare a field as transient for the purpose of Java serialization, yet persistent for the purpose of JDO, which is technically possible by specifying the Java `transient` keyword on the attribute, together with explicitly marking the field with `persistence-modifier="persistent"` in the JDO XML metadata descriptor.
- Change the domain model in order to make it "serialization-friendly" by attempting to "direct" associations in the "right" way, for the above example, make the relationship from Category to Book (as a Collection) instead of from Book to Category so that Book can be serialized

However all such approaches could rapidly defeat the purpose of an object-oriented transparent domain model. Using explicit data transfer objects and assemblers are thus usually a better choice for true a service-oriented architecture, albeit requiring extra coding. Attributes of such more complex data transfer objects should always be simple types such as String, double etc. or other data transfer objects, all created by an Assembler.

In the case of the above example, we could thus model a data transfer object (name it value object if you are more used to that terminology) for a persistent Book class that deliberately omits the link to `Category`, and instead contains less data, according specifically to the use case that requires this service. For example:

```
public class Category {
   //
   // JDO persistent-capable class that will be enhanced!
   //
   private Category parent;
   private List subCategories;
   private String name;

   // ...
}

public class Book {
   //
   // JDO persistent-capable class that will be enhanced!
   //
   private String author;
   private String isbn;
```

```
    private Category category;

    // ...
}

public class BookVO1 implements Serializable {
    private String author;
    private String isbn;
    private String category;  // String, not Category!

    public BookVO1(String bookAuthor, String bookISBN,
                   String bookCategoryName) {

        author   = bookAuthor;
        isbn     = bookISBN;
        category = bookCategoryName;  //  !
    }

    public String getAuthor()   { return author;   }
    public String getISBN()     { return isbn;     }
    public String getCategory() { return category; }

    // Note: Delibarately no setters yet!  See below.
}

// ...
   public Collection getBooks() {

   // Business logic method would not use retrieve() etc.
   //
      {
         BookVO dtoBook = new BookVO1( pcBook.getAuthor(),
                           pcBook.getISBN(),
                           pcBook.getCategory().getName());
```

In summary, the basic model is the same as has been recognized as EJB Best Practice for some time already, with explicit Data Transfer or Value Objects returned from inside Session beans.

Return trip ticket: Parameters

Another issue worth mentioning is what we may call the "return trip" of value objects from a client tier: Most real world applications will offer remote clients business methods related to updating data. In their simplest form, such methods could look like:

```
void updateBook(String bookISBNKey, String newAuthor);
```

However, given that we have already defined a data transfer class for books above, it could make sense to allow a client to change instances of these on the client (introducing setter methods) and send them back to a service:

```
public class BookVO2 implements Serializable {
    private String author;
    private String isbn;

    public String getAuthor()  { return author;  }
    void setAuthor(String _auth) { author = _auth; }

    public String getISBN()     { return isbn;     }
    void setISBN(String _isbn)  { isbn = _isbn;    }
}

public void updateBook(BookVO2 updatedBook) {
    // 1) Find persistent book
    // 2) Update persistent instance
}
```

Either way, the basic pattern is the same: First find the persistent object, then change it's attributes either to values passed as arguments to the business method, or from the DTO received as argument.

What differs is how to find the persistent object. Let's remind ourselves that we really do want to find an existing object, and change it. From update-type methods we never want to do something like:

```
Book pcBook = new Book(isbnCode);  // wrong here!
pm.makePersistent(pcBook);         // wrong here!
pcBook.setAuthor(newAuthor);
```

The above code snippet would invariable lead to the creation of a new persistent instance, not update any existing information. In order to find a persistent object to change, we

need to decide how the identity of the object to be changed is known to the service implementation. Several options exist:

- If the domain model uses JDO application identity and the key fields are part of the DTO, e.g. a persistent book class that would use the ISBN number as application identity and define a corresponding `BookID` class, the update code for the above example would look like this: (Issues around change of existing persistent object's application identity ignored for simplicity here.)

```
public void updateBook(BookVO2 updatedBook) {
    PersistenceManager pm = null;

    try {
        BookID objId;
        Book pcBook;

        pm = jdoPMF.getPersistenceManager();
        objId = new BookID(updatedBook.getISBN());
        pcBook = (Book)pm.getObjectById(objId, true);
        pcBook.setAuthor(updatedBook.getAuthor());

    } catch (Exception ex) {
        ejbCtx.setRollbackOnly();
        throw new EJBException(ex);
    } finally {
        try {
            if (pm != null && !pm.isClosed())
                pm.close();
        } catch (Exception ex) {
            // Log it
        }
    }
}
```

- If the domain model uses JDO datastore identity, the persistent book's object identity needs to be specified either via an extra parameter to the update-type method, or included in the DTO. Either way, this object identity should be transformed into a `String` for traveling across tiers, because using a parameter or member variable of type `Object` would introduce a dependency on the respective JDO implementation's class for datastore identity. So an extended DTO could look like this:

```
public class BookVO3 extends BookVO2 {
    private String oid;

    public BookVO3(String _oid) {
        oid = _oid;
    }

    public String getOID()  { return oid; }
}
```

While it is tempting at first to obtain the persistence-capable object's ID from within the DTO constructor, this would create a dependency on JDO in the DTO, thus on the client, and should be avoided. Instead, either use a separate assembler class, or pass the OID as parameter to the constructor as shown above, obtaining it via `PersistenceManager.getObjectId(pcBook).toString()` when constructing the DTO. The `PersistenceManager.newObjectId-Instance()` method then proves helpful for converting such an OID string back into a JDO identity in the service:

```
public void updateBook(BookVO3 updatedBook) {
      PersistenceManager pm = null;

      try {
          Object objId;
          Book pcBook;

          pm = jdoPMF.getPersistenceManager();
          objId = pm.newObjectIdInstance(Book.class,
                                updatedBook.getOID());
          pcBook = (Book)pm.getObjectById(objId, true);
          pcBook.setAuthor(updatedBook.getAuthor());

      } catch (Exception ex) {
          ejbCtx.setRollbackOnly();
          throw new EJBException(ex);
      } finally {
          try {
              if (pm != null && !pm.isClosed())
                  pm.close();
          } catch (Exception ex) {
              // Log it
          }
```

```
        }
    }
```

- If the object to update is part of a larger DTO, then none of the above is needed, as we'll already have an object reference to the respective persistent object, without having to go through ID objects. Care must be taken however to repeat the basic "find persistent instance, then update it" cycle for dependent DTO instances as well, in order to avoid assigning transient DTO objects to reference fields of persistent objects.

Various enhancements are possible around the approaches shown here. For example, optimizations to only update "relevant" persistent fields, usually a mechanism to recognize those which have actually been changed in data transfer object. The basic pattern will always stay the same however.

Web Services

The latest EJB specification standardizes how stateless session beans may be exposed as web services; this is termed a Web Service client view on a web service endpoint interface. So if you already have got stateless session beans, and an application server that supports the standard, this should purely become a matter of configuration at deployment.

However, to implement Web Services one neither necessarily needs EJB 2.1, nor EJBs at all. Several application server independent 3rd party tools exist to build Web Services around plain vanilla Java classes.

In both scenarios, data-transfer objects returned from services (and similarly service input arguments) are marshaled into XML data format according to a WSDL document in order to be packaged up in a Web service SOAP response.

MESSAGE-DRIVEN BEANS AND JDO

A message-driven bean is a server-side message consumer that has the following characteristics, according to the EJB specification:

- "Executes upon receipt of a single client message" and "is asynchronously invoked" by container on arrival of messages, never called directly by a client.
- "Can be transaction-aware"
- "Is relatively short-lived", and "is stateless". A typical EJB Container provides a scalable runtime environment to execute a large number of message-driven beans concurrently.

The EJB 2.0 specification supported only JMS message-driven beans. The EJB 2.1 specification extends the message-driven bean contracts to support other messaging types in addition to JMS.

Transaction demarcation for message-driven beans is similar to the Session beans as shown above. Again, both BMT and CMT are possible. In the case of CMT, only the `Required` and `NotSupported` transaction declarations make sense and are allowed.

Transaction management in the context of JDO for persistence operations is identical to what has been discussed earlier for session beans: Obtaining a `PersistenceManager` inside a CMT bean `onMessage()` method will ensure it is always correctly associated and enlisted in the respective J2EE transaction.

Example Code

The following code snippet shows how a message-driven bean might look:

```
public class OrderBooksBean
           implements MessageDrivenBean, MessageListener {

    private MessageDrivenContext ejbMsgCtx;
    private PersistenceManagerFactory jdoPMF;

    public void ejbCreate() {
    }

    public void setMessageDrivenContext(
              MessageDrivenContext messageDrivenContext)
                                  throws EJBException {

        ejbMsgCtx = messageDrivenContext;
        try {
            String jndiName = "java:comp/env/jdo/PMF";

            Context jndiInitCtx = new InitialContext();
            Object o = jndiInitCtx.lookup(jndiName);
            jdoPMF = (PersistenceManagerFactory)
                        PortableRemoteObject.narrow (o,
                          PersistenceManagerFactory.class);

        } catch (NamingException ex) {
            throw new EJBException(ex);
        }
    }
```

```
    public void ejbRemove() throws EJBException {
    }

    /**
     * Updates orders in persistent books.
     * Finds a book with the message.bookISBN, and
     * if it's price is lower than message.maxPrice,
     * then adds a new order for the message.clientNum.
     **/
    public void onMessage(Message message) {
        PersistenceManager pm = null;
        try {
            MapMessage mapMsg = (MapMessage)message;
            String bookISBN = mapMsg.getString("bookISBN");
            double maxPrice = mapMsg.getDouble("maxPrice");
            long  clientNum = mapMsg.getLong("clientNum");

            pm = jdoPMF.getPersistenceManager();
            BookID oid = new BookID(bookISBN);
            Book book = (Book)pm.getObjectById(oid, true);

            if ( book.getPrice() <= maxPrice ) {
                pcBook.newOrder(clientNum);
            }
            else {
                // Don't order... reject?
            }

        } catch (Exception ex) {
            ejbMsgCtx.setRollbackOnly();
            throw new EJBException(ex);
        } finally {
            try {
                if (pm != null && !pm.isClosed())
                    pm.close();
            } catch (Exception ex) {
                // Log it
            }
        }
    }
}
```

If persistent objects were to be included in messages, e.g. using `javax.jms.ObjectMessage`, then the points raised earlier in the context of session bean parameters (serializability or DTO, graph of objects, object ID as String, etc.) would all apply similarly.

Why use Message-Driven Beans?

Using Message Driven Enterprise JavaBeans offers advantages such as declarative transaction management, declarative messaging system parameters (e.g. JMS topics and destination) and most notably a generally scalable and clusterable container.

However, it is worth mentioning that it is technically not necessary to use a message-driven bean to listen to a JMS queue; a plain vanilla Java class might do just as well for given application. The point is: JMS and EJB are orthogonal to each other; so using JDO and JMS together does not necessarily require the use of EJB!

ENTITY BEANS AND JDO

After having discussed Session and Message-driven enterprise beans above, let's now turn to another type of enterprise bean, Entity Beans. As per the EJB specification, an entity enterprise bean has the following characteristics:

- "Provides an object view of data in the database"
- "Allows shared access from multiple users"
- "Can be long-lived (lives as long as the data in the database)"

Two different types of Entity Beans exist: Container Managed Persistence (CMP) and Bean Managed Persistence (BMP) Entity Beans. Both types of entity beans can be created and removed, and queried with a language named EJBQL. They also have an identity, their primary key. Does this all seem remarkably similar to what you have learnt about JDO so far to you? Well, you sure are not the only one! Indeed, the debate about when and whether to use Entity Beans versus JDO arose with the inception of JDO, and is an active and ongoing one.

One possible motive may be the idea to directly model remotely accessible persistent components. As we have seen earlier, JDO itself does not include any notion of remote invocation, but focuses purely on JVM local persistence issues. Entity beans may thus seem tempting at first for this purpose, instead of developing the required calling, pooling etc. infrastructure from scratch. However, most of today's EJB literature highly recommends using the so-called session façade pattern, wherein entity beans are accessed from other local enterprise beans only, and although technical possible, are not actually exposed to remote cli-

ents. An often cited and well documented reason for this architectural choice is performance issues with fine-grained remotely accessed Entity Beans. So if you use your Entity Beans only locally from Session and Message-driven beans anyway, why would you not rather forget about Entity beans all together and simply use the JDO API from within Session and Message-driven beans to implement persistence?

Another rationale could be existing Entity Beans that need to be reused. Given that these Entity Beans are usually persisted in a relatively straightforward relational schema when using Entity Beans, it is certainly worth at least evaluating whether migrating to an O/R mapping based JDO implementation that could reverse engineer the existing relational schema while preserving existing legacy data is a possibility. The chapter about JDBC of this book provides some more information on such a road.

A related possibility would arise should you have a project that uses existing Entity Beans which do not map data to and from a relational database, but uses another, e.g. an EIS-type or TP monitor, backend datastore. While JDO technically allows to and the market aims at providing JDO implementations for such datastores as well, it may not be available for your EIS of choice yet. It may be worth talking to both the EIS vendor about JDO, and independent JDO implementations about the EIS, in such a specific case. However, this is a somewhat theoretical possibility, because most real-world integration projects will join together using a higher-level API, likely service oriented; not on entity level. A similar and slightly more common scenario is if you have existing entity beans with BMP that do O/R mapping entirely through e.g. PL/SQL stored procedure calls rather than direct access to the underlying relational tables. Again, check if your JDO implementation for support of stored procedures.)

Last but not least, you could be on a project that does actually require an architecture with distributed and transactional persistent components. Read again, distributed transactional persistent objects, not distributable services. If you really are working on an application like this, the remotely accessed entity beans may be a fitting solution.

In short, with the advent of JDO it is hard to find good reasons on why to continue using Entity Beans instead of JDO for modeling and working with persistent objects in new J2EE projects.

Bean Managed Persistence (BMP) Entity Beans and JDO

Bean Managed Persistence (BMP) Entity Beans are components that respect the entity bean API contract but implement actual persistence code themselves within the BMP implementation class, instead of relying on 'transparent' declarative persistence handled by the EJB container, as is the case for Container Managed Persistence (CMP) Entity Beans.

Reasons for using BMP instead CMP enterprise beans could be limitations in (or in early containers simple lack of) Container Managed Persistence (CMP) Entity Beans, or non-relational data stores.

Should a specific application have a good reason to adopt such an architecture, it is technically certainly feasible to use JDO to implement the `ejbCreate`, `ejbRemove`,

`ejbActivate`, `ejbPassivate` as well as the `ejbLoad` and ejbStore methods that an BMP entity bean needs to provide. The JDO 1.0 specification outlines very roughly how this could be done in its chapter 16.2.1.[2]

However, as argued above, the real question in this context is not "How do you do this?" but much rather "Why would you do this?" The authors of this book believe that for most applications the answer simply is "You wouldn't do this!" and we thus do not provide examples of how to use JDO from BMP entity beans in this book.

Using JDO to implement CMP

Another issue is whether JDO could be used as the underlying technology to implement Container Managed Persistence (CMP) Entity Beans in a J2EE application server. The answer is definitely yes, JDO could be used for that purpose. This would however be an AppServer implementation decision, and is unrelated to the implementation of J2EE applications per se. This scenario will of course hide JDO features that have no CMP equivalent.

Indeed at least one (the SunOne AppServer) is reported to work like this. Interestingly, the O/R mapping toolkit that JBoss is using internally to implement CMP is also reported to likely become exposed for local (non-EJB!) persistence, although at the time of writing it is not clear whether this AOP-based service will be accessible via a JDO-compliant API.

Note also that using the standard JDO 1.0 API alone would lead to some minor issues complicating this slightly in practice; however vendor extensions could accommodate these: For example, seamless translation of some EJBQL to a JDOQL queries may be difficult due to the lack of bi-directional (inverse, managed) relationships as well as projections in JDO queries, see chapters 24.7 respectively 24.17 of the JDO 1.0 specification.

TO USE EJB OR NOT TO USE EJB?

After we have explored the details of JDO integration with EJB in this chapter, an often-asked question may be on your mind: Should you use EJB in a given application? The answer, as so often, is "it depends".

[2] If you are interesting in looking into this issue in more detail, be aware that the initial JDO 1.0 specification from spring 2002 mistakenly stated that "The `ejbActivate()` method acquires a `PersistenceManager` from the `PersistenceManagerFactory`....". This is wrong because the EJB (2.0) specification does not specify that `ejbActivate()` will be called within the context of an open transaction. Since persistence managers must be obtained within such a context in order to be bound to the J2EE transaction, acquiring the PM from the PMF should be done in the `ejbLoad()` method only, with the `ejbActivate()` method thus remaining empty. The v1.0.1 maintenance release of the JDO specification has corrected this detail.

Before proceeding, let's quickly recall what has been said earlier in this chapter already. Enterprise JavaBeans come in three flavors: Entity-, Session- and Message-driven beans. The authors of this book believe that JDO clearly offers a better alternative over EJB Entity beans, and advocate using JDO as primary persistence API within the service methods of Session and Message-driven beans. Alternatively, directly using JDBC or possibly a combination of JDO and JDBC may also be applicable.

However, as we have seen above and as readers familiar with the matter will know EJB-based development even with just two remaining bean types of interest represents a certain inherent complexity. The real question is thus: Should you use EJB Session and Message-driven beans at all?

Session and message-driven beans are components in the EJB architecture which essentially provides, once again: distributed component model, plus features such as security, transaction management and scalability. Another J2EE API provides similar features: The Servlets API. Servlets represent another kind of J2EE component architecture. It is worth comparing the two in more detail with respect to the features of interest to help in decision making whether EJB is appropriate for a given application.

- Component distribution: Does your application require fully distributed components which can be invoked from other remote components and dislocated plain old Java objects on other Java virtual machines? Then EJB session beans are probably a good choice. Or does your application mostly use collocated Plain Old Java Objects (POJO) "components" (i.e. well defined public interfaces with underlying implementation possibly spanning multiple classes that the component consumer should not directly access) with a few coarse grained services, including human-readable Web pages, offered to the outside world? Then Servlets may be enough![3]
- Service protocol: If remotely accessible services only need to be invoked via HTTP, then Servlets are sufficient.[4] If RMI and possibly IIOP for CORBA interoperability are a requirement, then EJB session beans are the obvious choice. If cross-firewall support is an issue, web services often make sense again, though. A similar argument applies if non-Java clients need access to the service, e.g. a .NET application. If performance is a concern, processing time for e.g. SOAP

[3] A Servlet-only based architecture is still "service-oriented" from an external system's point of view: Servlets offer HTTP-based (only) services, input arguments are the request parameters, and the returned documents contain the service result, either an HTML page if the service-consumer is a human, or as SOAP-based XML response (or simpler format, e.g. just plain text string) if the service-consumer is a machine.

[4] It is worth mentioning briefly that the Servlet API does technically provide for a non-HTTP Servlets, leaving the door open for support of other request/response style protocols in the future. This option is however almost never used in practice, and Servlet support in most application servers today really means HTTP Servlet support, only.

marshalling and unmarshalling overhead could be worth a thought, however. In short, it really depends on the big picture.

- Security: Security is a broad subject, encompassing Authentication, Authorization, Encryption and other topics. The relatively simple method-level declarative security model that EJB containers offer role-based authorization. Servlets can also be protected with role-based access control in the J2EE standard, albeit at an even cruder class-level only as they do not expose individual business methods by design. However, many real world applications today require more advanced security authorization anyway. Other security aspects such as authentication and encryption (SSL, via HTTPS in web container) are provided by both Web and EJB containers.
- Stateful architectures: Both the EJB and Servlet component models offer the possibility for stateful architectures via stateful session beans on one hand and the HTTP session variables on the other hand. However, if you do need a stateful distributed component architecture, the EJB model is possibly more appropriate.
- Message oriented architectures: If an application is essentially an asynchronous message-processing "sink", then relying on EJB 2.0 message-driven beans probably makes sense. The Servlet component model itself does not offer any comparable message awareness; however, plain vanilla Java classes can act as JMS destinations as well. Further discussion of this topic is outside of the scope of this book.
- Efficient multithreading and request dispatching: While an inherent feature of EJB containers, a web container as typically as well provides features like retrieving a thread from a thread pool, calling the correct method to handle the request, etc. This is thus not necessarily an argument in favor of the EJB component model. A more in-depth discussion on scalability with clustering follows below.
- Transaction demarcation: EJB offers declarative Transaction demarcation via CMT, for which there is no standard alternative in the Servlet component model. Servlets can however use explicit demarcation via JTA, a standard J2EE API that EJB also uses when operating in BMT mode. As we have seen earlier, JDO fully integrates with JTA and offers strong support for ACID transactions on persistent data, even when used outside of EJBs. (In a web-based architecture, including web-services, Transaction demarcation can often centralized in relatively few classes, e.g. in Servlets, a Servlet filter, MVC-type controller Action classes, or similar piece of codes.)
- Transaction context propagation: If your service is invoked remotely as part of a larger "value chain" i.e. in an Enterprise application integration (EAI) scenario, then EJB definitely makes sense, because the transaction context with the currently active transaction can be propagated across service method invocations of yours and other services and all related work can take place within one trans-

action. The Servlet API and Web service's standard do not (yet) offer any such possibility. EAI-type of applications will also frequently interact with existing EJB-based services, i.e. other session beans or possibly directly with Entity beans, and using fully EJB here definitely makes sense.

Concluding briefly, the first (distributed components) and the last (transaction context propagation) issues are often the decisive ones in favor of EJB if an application requires such.
For most other systems, most notably the common enterprise information systems (EIS) and productivity applications with mainly a web-centric presentation tier and straightforward transactional requirements, a Servlet-only based architecture is generally simpler and makes more sense.[5]

If while going through the above bullet points you conclude "mixed" results for your application, a "mixed" approach may well be what you need: EJB session beans for remote invocation via RMI, plus a few thin and simple Servlets for HTTP-based access. In this scenario, the Servlets may use the EJB 2.0 local interfaces to forward requests to collocated session beans, or both session beans and Servlets could possibly access Plain Old Java components.

Last but not least, a point needs to be made that this discussion centered specifically around the usage of the EJB Session and Message-driven beans and their APIs, and not J2EE application servers in general. Indeed the J2EE universe and a compliant Application Server offers many other APIs such as JMS, Servlets, JSP, JCA, JDBC, JNDI, JavaMail, JTA with a JTS-compliant transaction manager for possibly distributed transactions, JMX, etc. Modern J2EE application servers often also provide other services such as connection pooling and monitoring, hot deployment, clustering, cluster-related distribution and update features, security and user administration, generic monitoring and management features. All of these other API and features are also available to and integrate with JDO-based enterprise applications on a server that choose not to use the EJB component model.

Scalable JDO-based applications with clustering

If scalability requirements are the only reason to develop an application based on the EJB component model, then a second look at alternatives is time well spent. Following is an illustration of a typical non-clustered scenario, which will be extended next.

[5] The presentation and backend/business logic tiers can and should by the way still be clearly separated in the class design when the business service using JDO is collocated in the same VM as the Servlet tier instead of in a remote EJB tier. Such an approach is shown e.g. by the Sun Java Blue Prints Adventure Builder 1.0 (Early Access Release at the time of writing), albeit with direct JDBC instead of JDO, but the principle is the same.

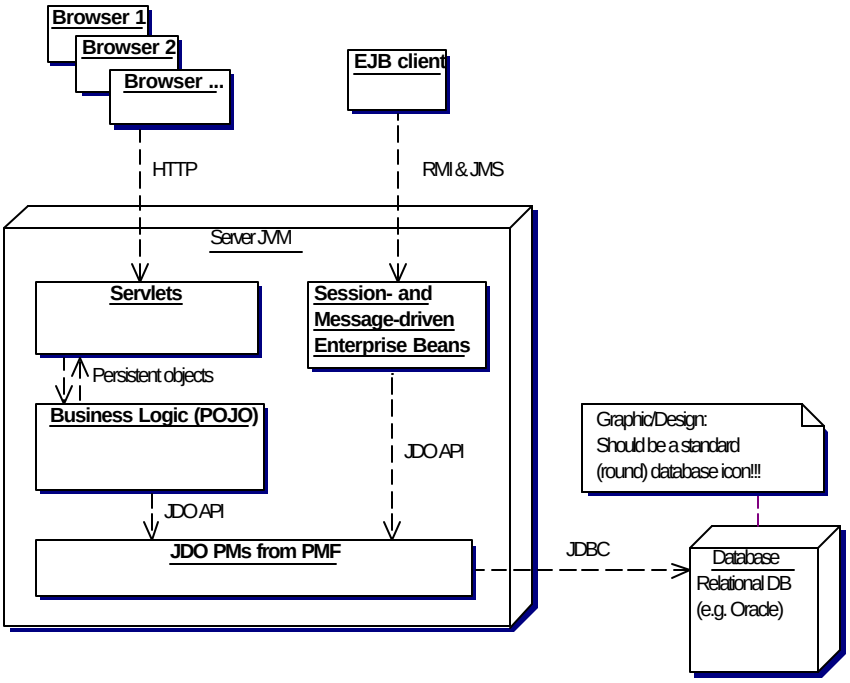

Figure 10-4 Simple system architecture with one server and no clustering

Scalability in this context usually refers to clustering mechanisms where requests can be redirected by software (or a hardware load-balancer) to different physical Java Virtual machines, often on physically different hardware boxes, depending on load and availability of servers. The goal of clustered system architectures is generally for one (or both) of the following reasons:

- High availability, a way to ensure "fail over" to a standby server
- Load balancing, distributing incoming requests for improved scalability

Such features are often provided by J2EE application servers and are thus far unrelated to JDO. Enter JDO into the picture, and with the extensive caching that JDO implementations usually perform a minor problem for clustering architectures: If nodes in a cluster are not aware of each other, and each keep an in-memory cache of persistent data, write access to a persistent object on one node will quickly render cached instances on other nodes out of date and thus invalid. Furthermore, concurrent modifications of identical instances on different nodes will arise as a related issue.

Two basic approaches exist to counter the issue and make clustering possible:

- Turn off caching altogether, and rely on datastore transactions only, with no JDO optimistic or locally optimized transactions. This resolves the problem, but implies a performance penalty that would generally largely outweigh the basic advantage of clustering tried to achieve in the first place. This configuration would, in a way, use only the underlying native datastore as "cache" instead of JDO.
- Use a JDO implementation that provides a distributed `PersistenceManagerFactory`-level cache synchronization feature.

Implementations of such cache synchronization features vary widely between JDO implementations, for those that offer such a feature at all, and include anything from simple broadcast UDP messages, to IP multicast-based solution, to a JMS-based one, or an HTTP based one, possibly using SOAP, or even RMI.

This feature is not standardized in the JDO 1.0 specification, but available from several JDO implementations. This type of feature is generally regarded as an implementation differentiator between vendors, and may not need standardization under the specification. Relevant implementations naturally do not interoperate. Configuration of a particular feature is highly dependant on the respective JDO implementation.

However, such a clustered deployment configuration should be completely transparent to application logic, and using an implementation-specific cache synchronization feature may not be a major portability issue, because changing to another JDO implementation will (ideally) only change the runtime deployment configuration, but no code.

In fact, the architecture used here is one of application-level clustering as opposed to individual object-level clustering: The objects representing business entities, the business logic, caches and the container are all collocated within one VM. Each node in such a cluster persists data directly to the underlying datastore. The nodes then use some sort of distributed caching coherency strategy akin to classical data replication among themselves. This architecture can perform particularly well for read-often, change-rarely data scenarios.

Note that clustering using this approach can be applied both when using JDO from within EJB and when using JDO directly from e.g. the web tier. The following illustration visualizes such an architecture:

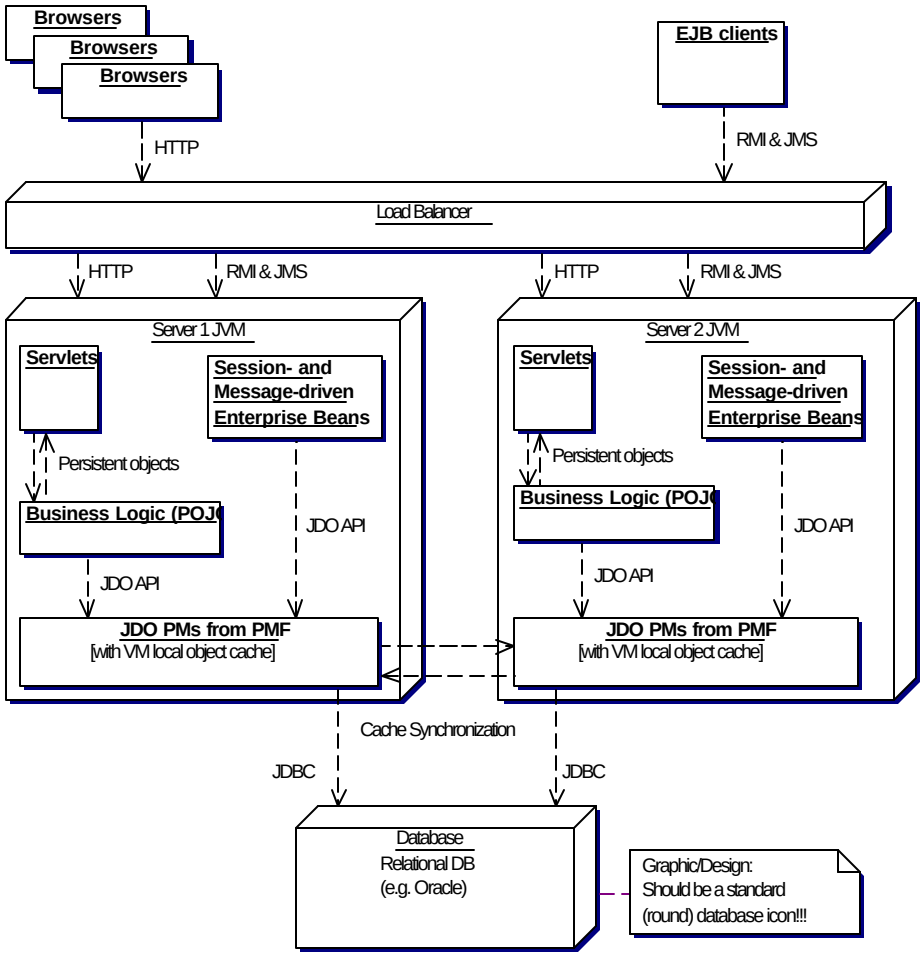

Figure 10-5 Cluster with two server nodes running JDO implementations with Cache Synchronization on one database server. Note how the JDO implementations on both servers directly access the datastore!

Round-tripping approaches

A technically slightly different approach to the one above is what is often known as “round-tripping” in its various forms. The basic idea is to provide a more or less lightweight client-side JDO `PersistenceManager` without access to the underlying datastore.

Such a local PM can transparently request copies of objects from a master server e.g. via a generic service with `getAllObjects(Query q, String[] fields)` and `updateObjects(Object[] changedObjects)` sort of methods, but provided by the JDO implementation and transparent to the application

Issues such as object graph diffing and instance reconciliation, relationship graph management and instance insertion ordering have to be addressed by such implementations.

In a less transparent and more explicit variation, an EJB session bean could send objects and partial object graphs that are copied to the remote machine, and then manipulated there and finally then sent back for resynchronization with the original persistent objects. Such features are not standardized in the JDO specification, but available in different shapes in some JDO implementations. The following illustration visualizes such an architecture:

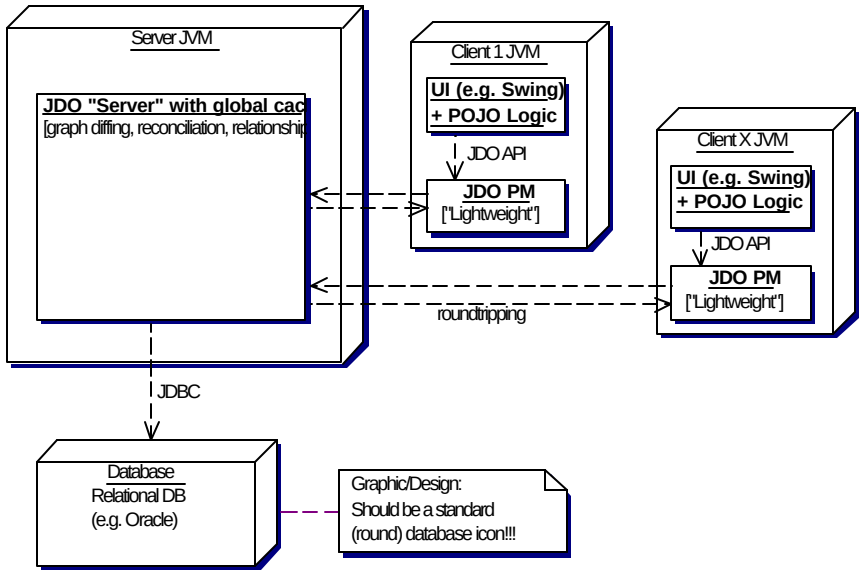

Figure 10-6 Architecture of "Distributed JDO" with a round-tripping like approach. Note how clients only interact with a server which accesses data store.

The architecture of such round-tripping (sometimes referred to as "Distributed JDO") approaches is generally more inline with optimistic/long running transactions. It could typically be well-suited for e.g. Swing-based fat GUI clients, if security and code distributions implications are not a concern. Such approaches are not service-oriented in the traditional sense. Scalability of architectures based on such features should be evaluated in more details before embarking fully on them.

SUMMARY

This chapter looked at the EJB component model and it's relationship to JDO. Various Implementation topics have been addressed with extensive source code examples. Higher-level conceptual and architectural issues have also been discussed.

In summary, if your application requires distributed and transactional persistent components, use EJB Entity Beans. In this case, if you are happy with the CMP entity bean implementations your container provides, JDO is not for you. If you are not, but your design does really require distributed and transactional persistent objects, BMP entity beans implemented using JDO may be the answer. For the rest of us, and that will be the vast majority, JDO provides a better alternative over EJB Entity Beans.

If you are looking at implementing a Service Oriented Architecture accessible from remote EJB or CORBA client fat clients, then EJB session beans are a great way to go. This architecture requires the passing of data transfer objects between tiers, as always, but JDO can be easily used to retrieve and update persistent state. The chapter has presented the practical details and options in such an approach.

If message-oriented features are of interest, the message-driven EJB component type can possibly be of interest. The chapter presented the integration of JDO in this sort of bean as well.

If however you are looking at implementing the average web-based enterprise application and would like a simple yet effective architecture, using JDO in plain Java objects that are VM local to the Servlet tier is a perfectly acceptable architecture. If web service support is of interest, this can be supported as well, without EJB.

As we have seen, even without the use of EJB a reasonable scalability can be achieved by using a clustered system architecture with a distributed cache synchronization with a JDO implementation that supports such an approach.